



# Lesson 7: Trees and Ensembles

Technologies for Artificial Intelligence

Fabio Antonini, Università degli Studi  
dell'Aquila

# Agenda

- A fourth family, built on a threshold
- Decision trees, and why one alone is mediocre
- Depth as the bias-variance dial, again
- Bagging and random forests
- Feature importance, and its limits
- Gradient boosting
- Choosing among trees, forests and boosting

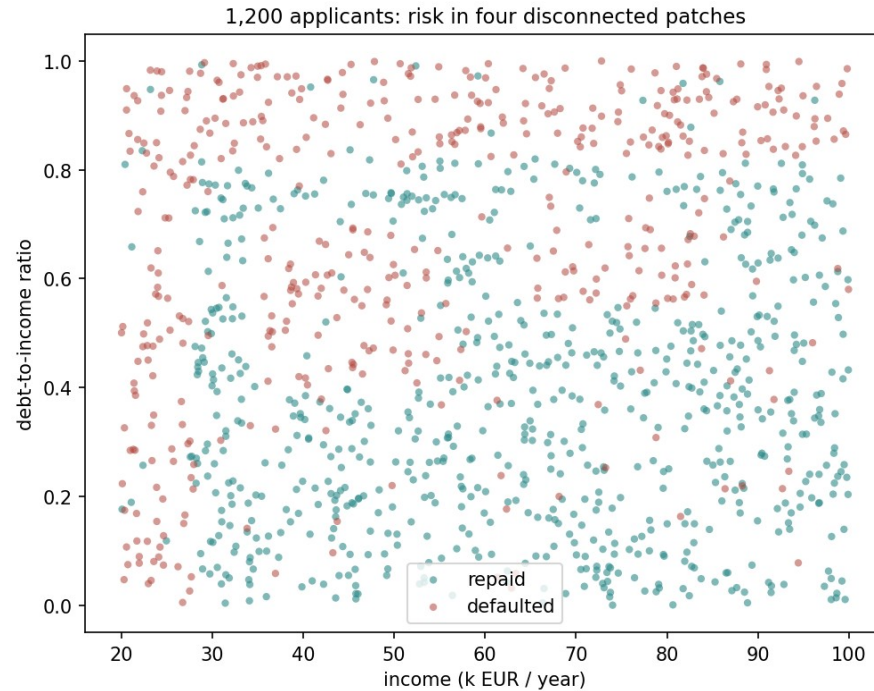
# Exercise 6, before we start

- What counts is the **defence of which family**, not the winning score
- The gap to watch for: a strong accuracy number with no check of *why* the assumption behind it held
- Today's models come with the same obligation

# 1,200 loan applicants, two figures each

- Income in thousands of euros a year, debt-to-income ratio
- Underwriting is a **checklist**, not a formula: an income floor, a debt ceiling, and two “stressed” combinations in between
- **7%** of labels are deliberately flipped; **38.7%** of applicants defaulted, so the majority baseline is **0.613**

# Four disconnected risky regions



# No straight line separates them

| Model                             | Cross-validated accuracy            |
|-----------------------------------|-------------------------------------|
| Always predict the majority class | 0.613                               |
| Logistic regression               | <b>0.748 <math>\pm</math> 0.030</b> |

# The whole algorithm, in one sentence

- Find the single (feature, threshold) split that makes the two resulting groups **as pure as possible**
- Apply it. Repeat on each group, independently
- Stop when a stopping rule is met
- Nothing is estimated, no coefficient, no gradient with respect to a parameter vector
- **Greedy**: whichever split helps most *right now*, with no lookahead

# How pure is a group of examples?

- A leaf that is all one class needs no more questions
- A leaf split evenly between classes is the worst case: a coin flip
- **Impurity** is a number that captures this, so “as pure as possible” has a target to maximise against

# Gini impurity

$$G = 1 - \sum_{c=1}^C p_c^2$$

# For two classes, one number

- $G = 2p(1-p)$ , where  $p$  is the fraction of one class
- **0** when the group is pure ( $p = 0$  or  $p = 1$ )
- **0.5**, the maximum, at  $p = 0.5$
- On the full 1,200 applicants,  $p = 0.387$  gives  $G = 0.474$

# The gain a split buys

- A candidate split divides a group into a left and a right child
- It is worth taking if the children are, **on average, purer** than the parent was
- “On average” is weighted by how many examples land in each child: a split that purifies nine points and dumps one troublemaker into its own leaf is not free

# The reduction in impurity

$$\Delta G = G_P - \left( \frac{m_L}{m_P} G_L + \frac{m_R}{m_P} G_R \right)$$

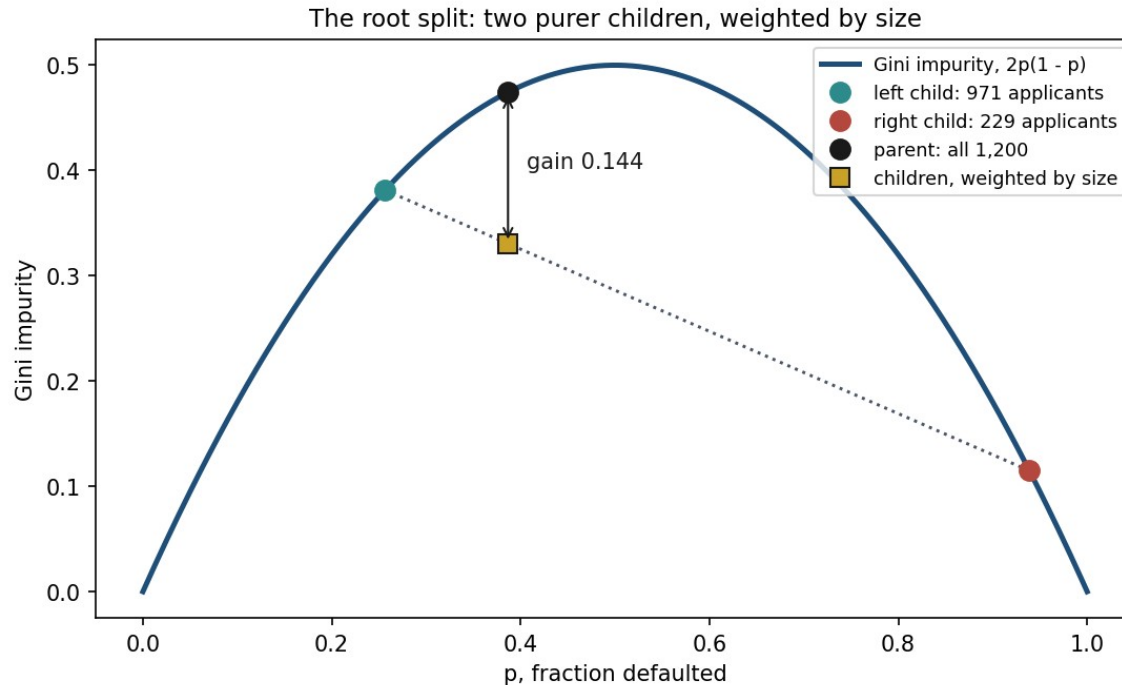
# Scanning for the best split

- For a continuous feature, scan **every midpoint** between adjacent sorted values as a candidate threshold
- Do this for **every feature**; keep the best pair
- Repeat the search inside each child
- **Greedy**:  $\Delta G$  is judged one split at a time, never jointly

# Worked example: the root split

- $p = 0.387$  defaulted, so the parent group's impurity is **0.474**
- Exhaustive search finds `debt_ratio <= 0.82` as the best root split
- Recognisably the **debt ceiling** the data was built with
- 971 applicants left (0.256 defaulted), 229 right (0.939): gain **0.144**

# The root split, on the Gini curve



# Checked against scikit-learn

- The from-scratch implementation agrees with scikit-learn's `DecisionTreeClassifier` to the **fourth decimal place** of resulting accuracy, at every depth tested
- Not a coincidence: both search the same space by the same greedy rule
- From here on, the notebooks use scikit-learn's implementation: the same algorithm, compiled, not a different one

# What greedy misses

- The search never asks “which pair of splits helps most together”: only “which single split helps most right now”
- A feature that only pays off **in combination** with a later split can be invisible to a shallow tree
- Exactly what happens to this dataset’s two interior “stressed” islands, covered next

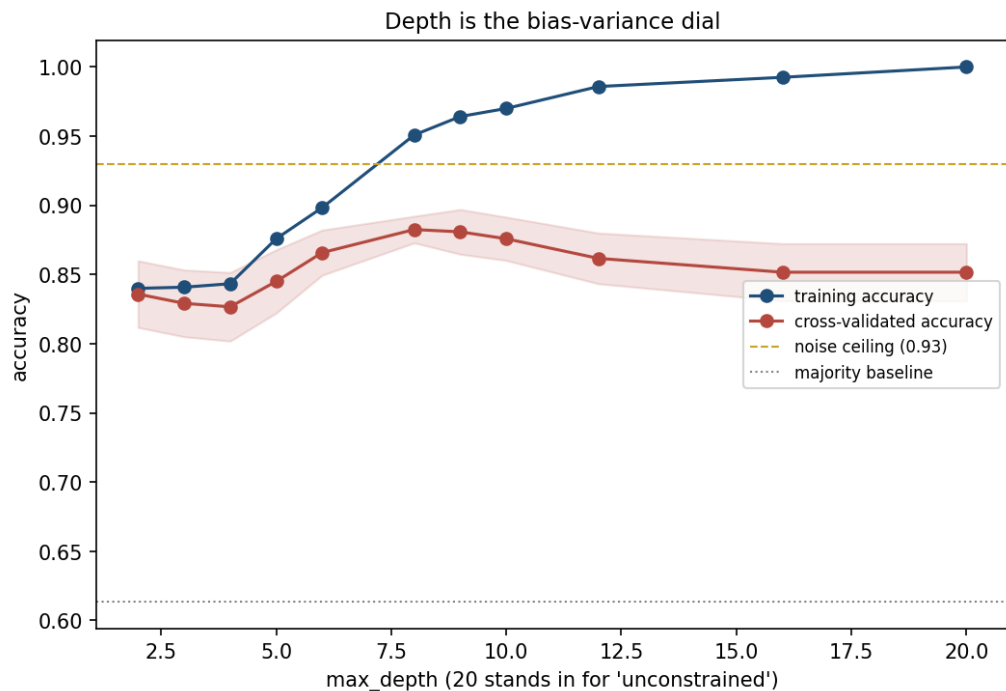
# Depth is the bias-variance dial

- `max_depth`: how many questions a tree may ask before a leaf must stop and vote by majority
- **Shallow tree**: few questions, only the coarsest structure. High bias, low variance
- **Deep tree**: enough questions to isolate almost any subset, down to single points. At full depth, training accuracy is **exactly 1.000 on any dataset**, including one with no structure at all

# Measured, at seven depths

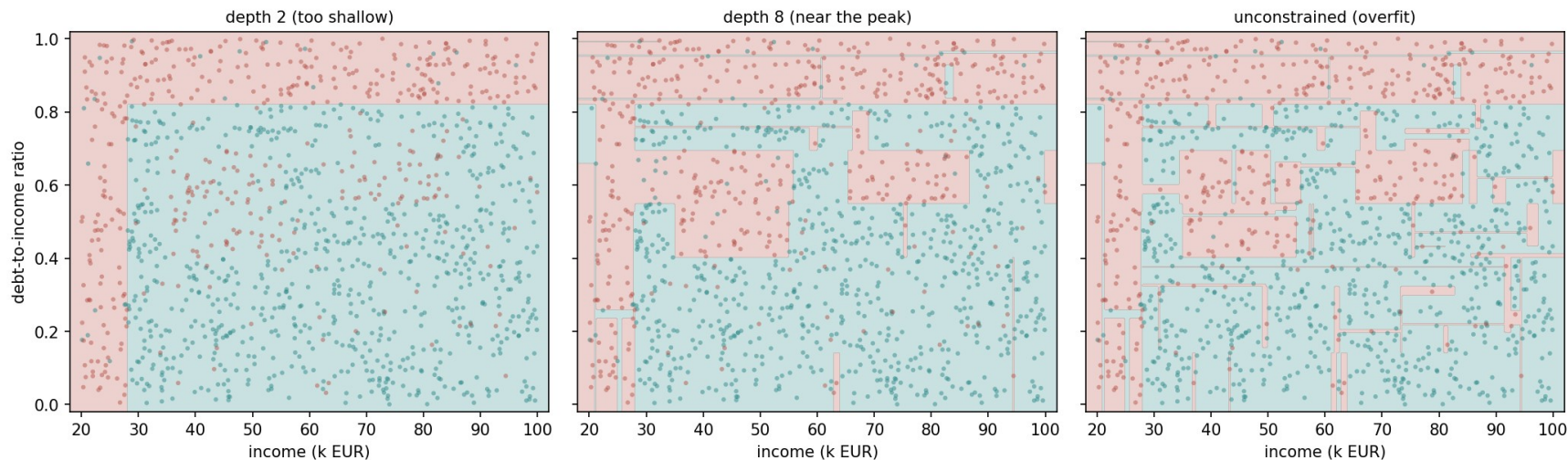
| max_depth     | Training accuracy | Cross-validated accuracy            |
|---------------|-------------------|-------------------------------------|
| 2             | 0.840             | $0.836 \pm 0.024$                   |
| 4             | 0.843             | $0.827 \pm 0.025$                   |
| 6             | 0.898             | $0.866 \pm 0.016$                   |
| <b>8</b>      | 0.951             | <b><math>0.882 \pm 0.010</math></b> |
| 12            | 0.986             | $0.862 \pm 0.018$                   |
| 16            | 0.993             | $0.852 \pm 0.021$                   |
| unconstrained | <b>1.000</b>      | $0.852 \pm 0.021$                   |

# The curve bends downward



# The same rule, three depths

The same rule, three depths



# Reading a tree, and the knob that keeps it small

- `max_depth` grows the tree **breadth-first**: every leaf at level  $d$  splits before any leaf reaches  $d+1$ , useful or not
- `max_leaf_nodes` grows it **best-first**: always expanding whichever leaf offers the largest  $\Delta G$
- With `max_leaf_nodes = 9`: **90.8%** training accuracy, every threshold recognisable: 28 the income floor, 0.82 the debt ceiling

# A tree that doesn't stay still

- Two trees at `max_depth = 6`, fit to independent 65% resamples of the same 1,200 applicants
- The debt ceiling, the strongest split, agrees to within **0.001**
- The trees grow **31 and 33 leaves**, and **disagree on 11.7%** of predictions on the full dataset
- The strongest split is stable; the many weaker ones are not, and they decide most of a tree's shape

# Notebook 1, live

- Decision trees from scratch, checked against scikit-learn
- Depth swept, the two boundaries drawn, and the instability measured

# Break

- Twelve minutes

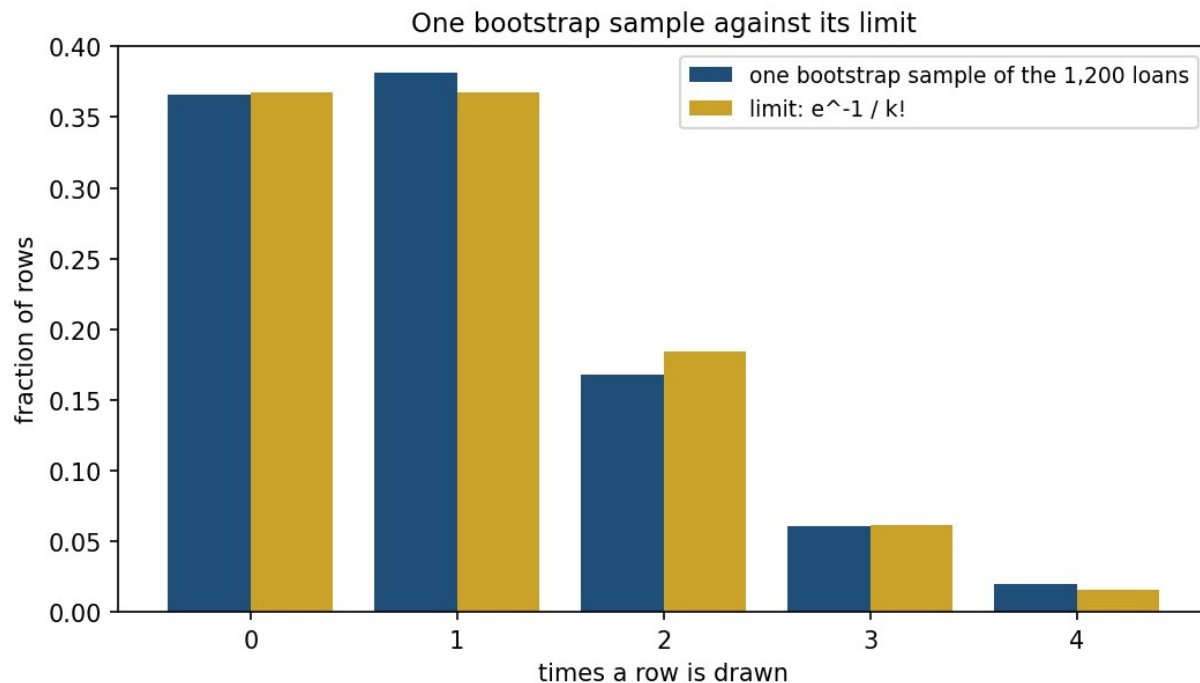
# Bagging: averaging away the instability

- A tree's mistakes differ across resamples, as just measured, so **many trees, averaged** should partly cancel out
- Draw a **bootstrap sample**:  $m$  rows chosen **with replacement** from the  $m$  training rows
- Fit a tree to it. Repeat `n_estimators` times; predict by **majority vote**
- The tree algorithm itself is unchanged, only what surrounds it is new

# How much data one bootstrap sample sees

- Each bootstrap sample is drawn from the same  $m$  rows, with replacement
- About **63.2%** of the distinct rows are drawn at least once
- The remaining **36.8%** are left out entirely: called **out-of-bag (OOB)**
- At  $m = 1,200$ : a single simulated draw left out 36.6% of rows, close to the limit

# One bootstrap sample, counted



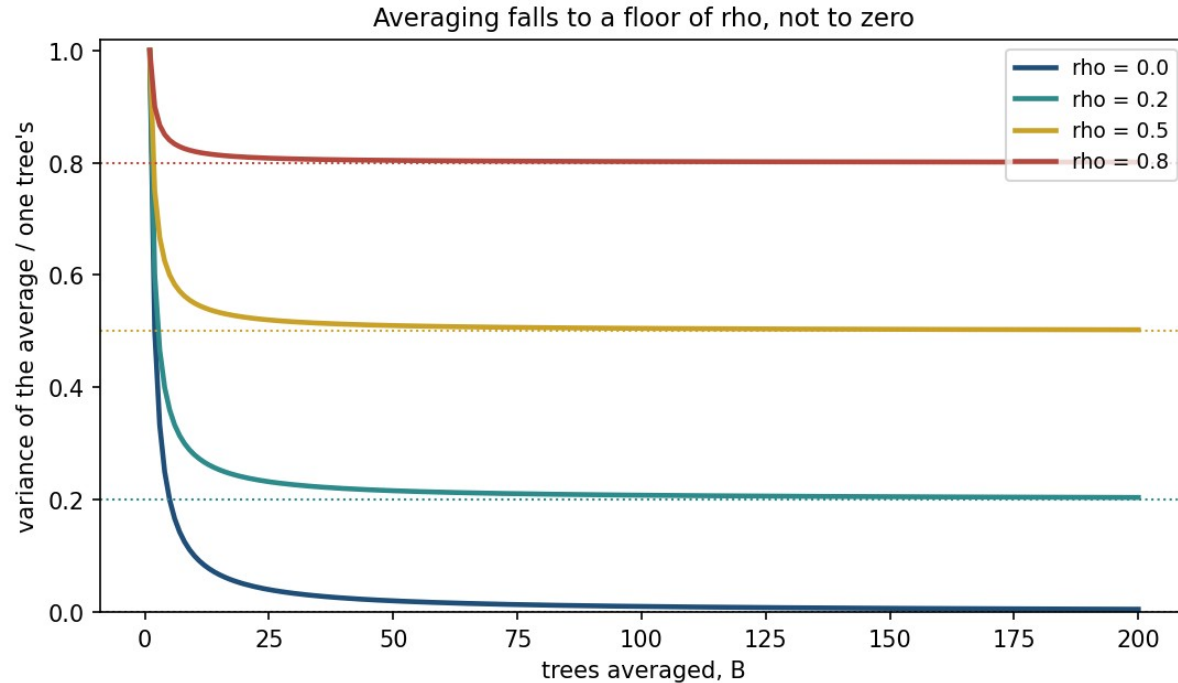
# Out-of-bag: free validation

- Every out-of-bag row is validation data for the tree that missed it
- Averaging them gives the **OOB score**, at no extra split
- A 300-tree forest: OOB **0.9117**, 5-fold cross-validated **0.9117  $\pm$  0.021**
- That match to four decimals is **this seed's luck**: over twelve seeds the gap is 0.003: agreement to within their own noise, not to every decimal

# Why averaging reduces variance

- Model each tree's prediction as noisy, with variance  $\sigma^2$ , and let  $\rho$  be the correlation between any two trees
- Averaging  $B$  trees drives variance toward a **floor** of  $\rho\sigma^2$ , **not zero**: adding trees past that point buys almost nothing
- Averaging **cannot fix bias**: unbiased-but-noisy trees average to unbiased; trees sharing a systematic error keep that error exactly

# Averaging stops at a floor



# Bagging, measured

| Model                     | Mean test accuracy | Standard deviation |
|---------------------------|--------------------|--------------------|
| Single unconstrained tree | 0.856              | 0.0185             |
| Bagging, 100 trees        | <b>0.902</b>       | <b>0.0153</b>      |

# Random forests: decorrelating the trees

- The variance floor is  $\rho\sigma^2$ : lowering it further means making the trees agree **less**
- Bagged trees still see **every feature** at every split, so a strong feature pulls most of them towards splitting on it first
- A **random forest** restricts each split to a random subset of `max_features` (default  $\sqrt{n}$ , rounded down): different trees see different features at the same point, lowering  $\rho$

# Random forests, measured

| Model                           | Mean test accuracy | Standard deviation |
|---------------------------------|--------------------|--------------------|
| Single unconstrained tree       | 0.856              | 0.0185             |
| Bagging, 100 trees              | 0.902              | 0.0153             |
| <b>Random forest, 100 trees</b> | <b>0.908</b>       | <b>0.0137</b>      |

# Feature importance: what the forest claims to tell you

- A random forest reports **feature importance**: the impurity reduction each feature is responsible for, summed over every tree and split
- Tempting to read as “the forest tells you which features matter” - and with only the two real features here, it does exactly that
- The question: what happens once most of the columns carry **nothing**?

# More columns, more noise

| Noise columns added | Total columns | Cross-validated accuracy | Importance on noise |
|---------------------|---------------|--------------------------|---------------------|
| 0                   | 2             | 0.912                    | 0.0%                |
| 5                   | 7             | 0.874                    | 33.9%               |
| 20                  | 22            | 0.837                    | <b>54.5%</b>        |

# Notebook 2, live

- Bagging and random forests, from scratch and against scikit-learn
- The out-of-bag score checked against cross-validation
- The noise-column experiment, run live

# Gradient boosting: correcting mistakes in sequence

- Bagging and random forests build trees **independently** and average at the end
- **Boosting** builds trees **in sequence**: each new tree is fit specifically to correct what the trees so far got wrong
- Each addition is a shallow tree: depth 2 or 3, a **weak learner**
- The learning rate  $\alpha$  sets **how much** of each new tree's correction is actually applied

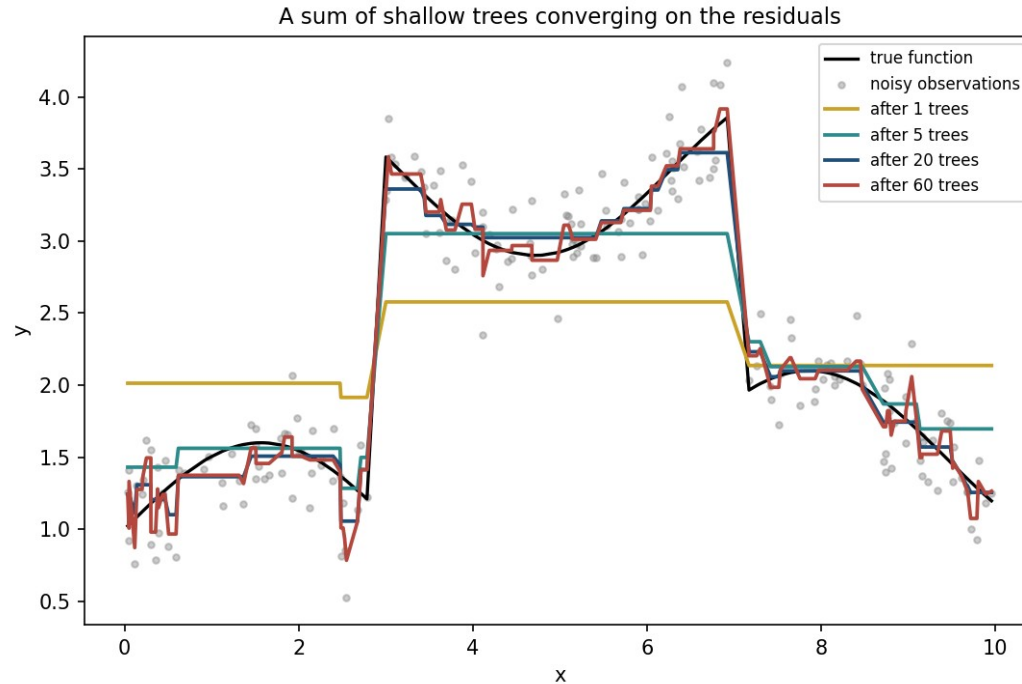
# Building the ensemble, one tree at a time

$$F_0(x) = \bar{y}, \quad F_t(x) = F_{t-1}(x) + \alpha \cdot h_t(x)$$

# What each tree is fit to

- For squared-error regression, “what remains unexplained” has an exact name: the **residual**,  $y - F_{t-1}(x)$
- This is a special case of a more general idea: **functional gradient descent**:  $h_t$  approximates the negative gradient of the loss with respect to  $F_{t-1}$
- For squared error, “fit the residual” and “fit the negative gradient” coincide exactly

# A sum of shallow trees converging on the residuals



# Reading the picture

- **1 tree**: mean squared error (MSE) against the true function 0.395, barely moves the flat guess
- **5 trees**: MSE 0.068, the coarse shape appears
- **20 trees**: MSE **0.012**, the ensemble's best point
- **60 trees**: MSE rises back to 0.018, tracking noise, not signal

# Classification: the same idea, a different gradient

- The label  $y$  is 0 or 1; the ensemble's output becomes a probability,  $p = \sigma(F)$
- The loss is **log-loss**: lesson 4's loss for logistic regression
- The negative gradient, the **pseudo-residual** each tree fits, is  $y - p$
- True label minus predicted probability: lesson 4's error term exactly

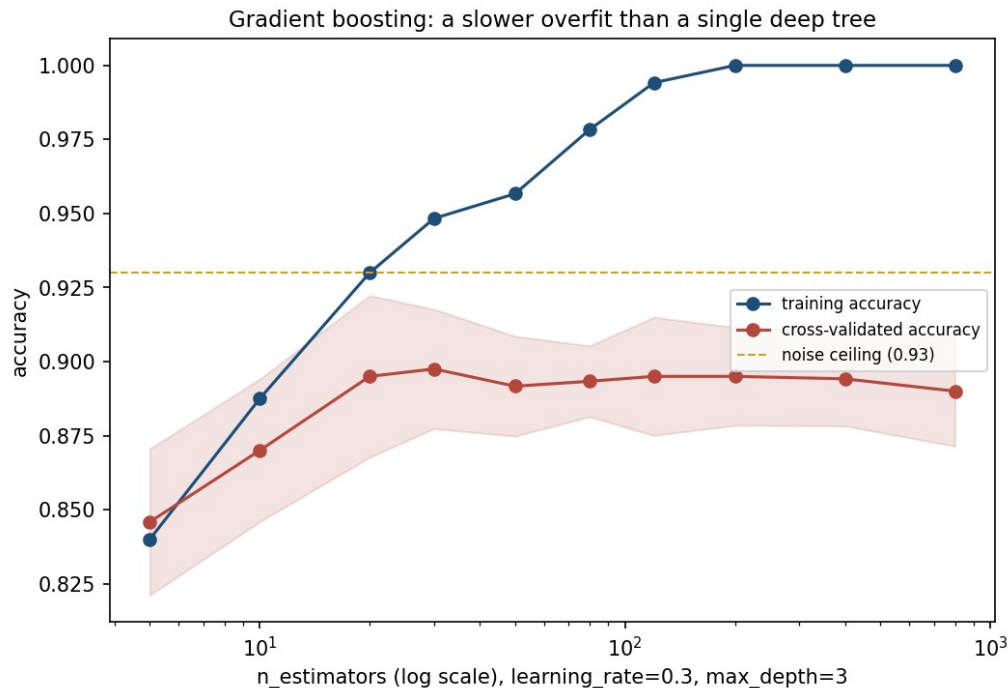
# On the loan data

- `GradientBoostingClassifier`, defaults: **100 trees**, learning rate  $\alpha = 0.1$
- Reaches  **$0.902 \pm 0.020$**  cross-validated accuracy
- Already ahead of any single tree from earlier in the lesson

# Boosting never stops fitting on its own

| n_estimators ( $\alpha = 0.3$ ) | Training accuracy | Cross-validated accuracy            |
|---------------------------------|-------------------|-------------------------------------|
| 5                               | 0.840             | $0.846 \pm 0.025$                   |
| 10                              | 0.888             | $0.870 \pm 0.024$                   |
| 20                              | 0.930             | $0.895 \pm 0.027$                   |
| <b>30</b>                       | 0.948             | <b><math>0.897 \pm 0.020</math></b> |
| 80                              | 0.978             | $0.893 \pm 0.012$                   |
| 200                             | 1.000             | $0.895 \pm 0.017$                   |
| 800                             | 1.000             | $0.890 \pm 0.019$                   |

# Training climbs to 1.000 and stays; the honest curve turns over first



# Fixing 120 trees, varying only the learning rate

| Learning rate $\alpha$ | Training accuracy | Cross-validated accuracy            |
|------------------------|-------------------|-------------------------------------|
| 0.02                   | 0.888             | $0.872 \pm 0.025$                   |
| 0.05                   | 0.914             | $0.897 \pm 0.022$                   |
| <b>0.10</b>            | 0.939             | <b><math>0.902 \pm 0.022</math></b> |
| 0.30                   | 0.994             | $0.895 \pm 0.020$                   |
| 1.00                   | 1.000             | $0.878 \pm 0.020$                   |

# Notebook 3, live

- Boosting from scratch on residuals, and against scikit-learn
- The classification pseudo-residual, and the learning-rate sweep

# Every method, on the same five folds

| Model                                   | Cross-validated accuracy |
|-----------------------------------------|--------------------------|
| Majority baseline                       | 0.613                    |
| Logistic regression                     | 0.748                    |
| Single tree, unconstrained              | 0.852                    |
| Single tree, <code>max_depth = 8</code> | 0.882                    |
| Gradient boosting, 30 trees             | 0.897                    |
| Bagging, 100 trees                      | 0.904                    |
| <b>Random forest, 100 trees</b>         | <b>0.911</b>             |
| <i>noise ceiling</i>                    | $\approx 0.93$           |

# Three findings

- Every ensemble lands within a point and a half of the others: **which strategy** matters far less than **whether you ensemble at all**
- The tuned single tree is close behind, 0.882 against 0.911, and the only model here a person can read
- The **unconstrained tree is the worst model on the list**, despite being the most flexible one

# How to choose, in practice

| Situation                                                        | Reach for                                            |
|------------------------------------------------------------------|------------------------------------------------------|
| The decision must be explained to the person it affects          | A single tree, depth-tuned, or nothing on this list  |
| Tabular data, accuracy matters most, no explanation required     | A random forest, as a strong and low-effort default  |
| Training time matters, or the signal needs sequential refinement | Gradient boosting, tuned on a validation set         |
| Very high-dimensional, few informative columns                   | Any tree method, cautiously                          |
| Rows only, and OOB score suffices for validation                 | A random forest, no explicit train/test split needed |

# 54% of the importance landed on noise

- 20 noise columns, 2 real features: not a small effect, and not a bug
- The same restriction that **decorrelates** the trees occasionally hands a split to noise instead
- Averaging reduces **variance of the prediction**; it does **not** guarantee small importance for irrelevant columns
- The fix: cross-check against permutation importance on held-out data

# What to take away

- A tree splits greedily on Gini impurity, matching scikit-learn
- Depth is the bias-variance dial: accuracy peaked at **depth 8**
- Bootstrap resampling leaves **36.8%** of rows out-of-bag: free validation
- **54% of a random forest's importance landed on 20 noise columns**, today's number

# Homework

- **Exercise 7**, discussed at the start of **Friday 20 November**
- `Exercises/07_trees_and_ensembles.md`